Candidate Selection and Review Gates for Wrong Writes and Missed Updates in User-Controlled LLM Memory

Radha Mohan Pattanayak
Associate Professor, SCOPE
VIT-AP University
Amaravati, India
radhamohan.pattanayak@vitap.ac.in

Soubhagya Swastik Dhal
B.Tech (Computer Science & Engineering)
VIT-AP University
Amaravati, India
soubhagya3696@gmail.com

Basant Kumar
B.Tech (Computer Science & Engineering)
VIT-AP University
Amaravati, India
gbasant201036@gmail.com

Mehal Mathur
B.Tech (Computer Science & Engineering)
VIT-AP University
Amaravati, India
mehalmathur24@gmail.com

Aditya Kumar Jena
B.Tech (Computer Science & Engineering)
VIT-AP University
Amaravati, India
adi.jena1904@gmail.com

***Abstract*—Assistants that remember facts about their user ask for approval before changing what they believe. We study what such an approval gate protects and what it cannot, in AURA, whose language-model extractor proposes add, update, retire and forget operations over a retrieval-selected list of candidate memories. First, an update can succeed only if its target memory is in the extractor's prompt, so a gate that acts afterwards cannot repair missed updates; in simulation it removes 94–98% of the true facts that automatic application destroys but leaves stale facts unchanged. Second, we define final-prompt reachability and measure its drivers on a new test set of 40 personas and 15 selection variants. With 350 unrelated memories the default selection reaches 0.667 of targets, an adaptive selection 0.845, and a plain list of the 24 best matches 0.849 (Holm $p = 0.860$ for the difference); keyword and fused rankings do no better; a cross-encoder reranker reaches 0.887, and 7.7% of targets stay out of reach of every method. Third, with two real Gemini extractors the target was missing in 25 and 22 of 136 change turns under the default selection and in none under the adaptive one, and stale facts under the gate fell from about 1 to 0 per persona, at about 35% more prompt tokens. A six-persona pilot shows the plain list matching the adaptive selection end to end. On all 72 LongMemEval knowledge-update questions neither selection changed accuracy. Code, test sets and raw results are released.**

***Index Terms*—long-term memory, conversational assistants, large language models, human-in-the-loop, memory update, candidate selection, retrieval, evaluation.**

I. Introduction

Large language model (LLM) assistants that remember facts about their user across conversations must decide, after every message, whether to add a fact, change one they already hold, or drop one. Related work has contributed in four ways. Benchmarks measure long-term memory through question answering: LoCoMo uses very long agent–agent conversations [1], and LongMemEval tests five memory abilities, among them reasoning about updated facts [2]. Retrieve-then-update pipelines such as Mem0 extract candidate facts, retrieve the most similar stored memories and let a language model add, update, delete or leave memory unchanged [3]. Learned and agentic updaters train a policy to write, revise and prune memory (TrustMem [4], AgeMem [5]) or to rewrite a bounded notes field (Supersede [6]), and retrieval-centred designs organise or filter memory for answering (REALM [7], MemGate [8]). Governance work proposes consistency checks and permission controls before a memory is stored [9] and shows how errors entering through memory updates persist and compound [10], [11]. Together they show that update failures and stale facts are measurable problems (Sec. II).

Failures that a gate cannot see. The governance studies check or verify what a memory writer *produces*, and the trained updaters reduce errors in the transitions they emit. A direct form of such a check is to ask the user: changes to existing memories become proposals that the user accepts or rejects, and old values are kept as history. None of these works separates a wrong write from a missed update, and none asks whether the record that must change was ever placed before the updater. In a retrieval-based design the extractor can change only the memories in its prompt, so an update it never produces cannot be proposed, and therefore cannot be approved. We close this gap with the following contribution.

- **Failure isolation.** We separate wrong writes from missed updates and show where each is decided. Because AURA discards any operation that names a memory the extractor was not shown, the success of a change factors exactly into whether its target reaches the prompt and whether the change is then proposed and accepted (Sec. III); the gate acts only on the second factor. Under a controlled

error model it cuts destroyed true facts by 94–98% but leaves stale facts essentially unchanged (Sec. VI).

Retrieval that is not aimed at updates. Mem0 retrieves the most similar memories for each new fact, REALM and MemGate retrieve or filter memory for answering, and Supersede avoids retrieval by rewriting whole notes. Of the works we read, none measures whether the record that a requested change must target is in the updater's prompt, or varies how it is chosen, and the benchmarks grade answers rather than the prompt the updater saw. Recall@k of the retriever is the closest quantity, but it ignores the minimum score and the recent-memory slots that decide the final prompt. We contribute the following.

- **Reachability and its drivers.** We define reachability on the final extractor prompt for known change events, distinguish it from retrieval recall, and measure it over store sizes, message forms and kinds of change on two test sets (Sec. VIII). A controlled study on a new test set of 40 personas with 15 selection variants, including same-size, keyword (BM25) and fused baselines, a cross-encoder reranker, and change messages held out from the selection's cue words, shows that removing the minimum score and searching per sentence each add about 0.1, that the adaptive selection reaches the same share as a plain larger list while showing 19% fewer candidates, that keyword and fused rankings do no better than the plain list, and that about one target in thirteen stays out of reach of every method tested (Sec. IX).

End-to-end evidence and its limits. The benchmarks and most updater studies report answer accuracy or transition-level scores for one configuration. They do not report, with the update gate in place, how many stale facts remain, how many prompts the user must answer, or what a better selection costs in tokens, and none of those we read compares a remedy with a plain larger list or with an oracle that always shows the target. We contribute the following.

- **Real-model evidence with costs.** With two Gemini extractors we evaluate the adaptive selection (F2) and a re-proposal rule (F1) in a paired ablation: with F2 (run together with F1) no change target was missing and the stale facts under the gate disappeared, at about 35% more prompt tokens; F1 alone showed no measurable benefit (Sec. X–XI). A six-persona pilot adds a plain list of 24 and an oracle-target condition (Sec. IX-A), and a check on all 72 LongMemEval knowledge-update questions is reported with its limits (Sec. XII). The harness, both test-set generators and all raw results are released.

The rest of the paper is organised as follows. Section II places the work among related studies, Section III defines the problem and the quantities we measure, Section IV describes the memory system, and Section V the evaluation. Section VI shows what the gate protects, Sections VII and VIII how often the target reaches the extractor, and Section IX which part of candidate selection matters. Sections X and XI report the ablation and the real Gemini extractors, and Section XII the LongMemEval check. Sections XIII–XV discuss implications, limits and future work. We deliberately keep claims narrow: the personas are synthetic and English, the real-model samples are small, the user is simulated, and no user study was run (Sec. XIV).

II. Related Work

Benchmarks. LoCoMo evaluates very long agent–agent conversations with question answering, summarisation and multimodal tasks [1]. LongMemEval benchmarks chat assistants on five long-term memory abilities, among them reasoning about updated facts, and splits memory design into indexing, retrieval and reading [2]. We use LongMemEval's knowledge-update questions as an external check (Sec. XII).

Retrieve-then-update pipelines. Mem0 extracts candidate facts from a conversation, retrieves the most similar stored memories for each, and lets a language model choose to add, update, delete or leave memory unchanged [3]. AURA's default pipeline follows the same pattern, with two differences: changes to existing memories wait for the user, and old values are kept. Our results therefore bear on any pipeline of this shape: whatever the retriever does not place before the model cannot be updated.

Learning reliable memory updates. TrustMem trains a memory agent that reads a chunk of input together with the current memory and emits write, revise and prune actions [4]. A frozen language model, the Memory Transition Verifier, scores each transition for *coverage* (information from the current chunk is kept), *preservation* (valid prior memory is not damaged) and *faithfulness* (new content is supported); it sees the chunk, the actions and the memory entries they touch or retrieve. The scores drive Transition-Ranked GRPO, and on MemoryAgentBench, HaluMem and the Mem-α validation set the method reduces transition-level omission, corruption and hallucination by 40.1%, 79.1% and 50.0% against the strongest baseline for each. TrustMem's omission is information from the new input that is not stored; the failure we isolate is different and earlier: the stored record that must change is absent from the updater's input, so no transition on it can be generated. AgeMem exposes add, update and delete operations on long-term memory and retrieve, summary and filter operations on short-term context as tools of one policy, trained in three stages with a step-wise GRPO, and evaluates on ALFWorld, SciWorld, PDDL, BabyAI and HotpotQA [5]; which records the policy sees when it updates is part of what it learns, and is not measured separately. Supersede maintains a bounded notes memory that the agent rewrites after each session; on the 78 knowledge-update questions of LongMemEval, bounded memory lowers accuracy from 92% to 77% for a frontier model (paired McNemar $p < 0.005$) and from 82% to 63% for a smaller one, the gap grows with conversation length rather than shrinking with more memory, and GRPO fine-tuning of a 3B model raises held-out supersession accuracy from 9.0% to 16.7% in a single run [6]. Because the whole notes field is rewritten, no retrieval step stands between memory and updater there; the gap Supersede measures is one of maintenance, while ours is one of visibility. Both papers support the premise that stale facts are a real, distinct failure, and Supersede's finding that bounded memory stays below full context matches our

LongMemEval observation. PAMU tracks evolving preferences as a vector, combining sliding-window averages and exponential moving averages with a change-detection signal, and evaluates on LoCoMo [12]; it updates a preference representation, not discrete stored facts.

Retrieval that shapes memory and access to it. REALM organises memory as a heterogeneous graph, retrieves evidence with adaptively composed graph-search operations and reconsolidates memory from retrieval feedback, reaching 75.97% on LoCoMo and 65.11% on LongMemEval [7]. Retrieval there serves answering and drives reorganisation; it is not used to choose the record that a requested change must target. MemGate places a 9M-parameter, query-conditioned gate between a vector store and the model that decides which retrieved memories are admitted [8]; on a personal-agent environment with GPT-4o-mini it lowers cross-domain leakage from 27.0% to 3.5% and jailbreak success from 16.8% to 4.4%, while LoCoMo F1 rises from 38.9 to 40.8. MemGate filters what reaches a response; a filter placed after candidate selection can only remove candidates, so it cannot raise the reachability we study, although it could make larger candidate lists safer.

Governance and error propagation. Lam et al. propose a governance framework that separates memory updates from task execution and applies consistency checks and permission controls before storage [9]. Xiong et al. show that adding and deleting stored experiences changes agent behaviour and that errors can compound through retrieval [10]; Xiao et al. trace how errors entering through memory updates persist over turns [11]; Chu surveys security threats to agents by layer [13].

Positioning. These studies establish that reliable long-term memory needs more than storing and retrieving similar text, and that update failures and stale facts are measurable problems. The present work does not claim to introduce update reliability or retrieval before updating. It examines one narrower condition in user-controlled, retrieval-based updating: the stored record that must change may be absent from the updater's prompt, in which case no approval step can repair the miss. We measure this condition directly, separate it from the gate's effect, and test which parts of an inexpensive, inference-time change to candidate selection address it. TrustMem, AgeMem, Supersede, PAMU, REALM and MemGate were read in full for this paper; the other works were read at the level of their abstracts. A more systematic search for work on update-target retrieval may find closer studies, and we make no priority claim.

III. Problem Definition

Let U be the set of change events in a conversation: user messages that require an existing memory to be updated, retired or forgotten. For an event u, let m_u^* be the stored record that must change and C_u the set of memories in the extractor's final prompt. In AURA, $C_u = S_k(x_u) \cup R_8$, where $S_k(x_u)$ are the at most k best search matches for the message x_u that pass the minimum score and R_8 are the eight most recently confirmed memories. We define *final-prompt reachability* as

$$\mathrm{CR} = \frac{1}{|U|} \sum_{u \in U} \mathbf{1}\left[m_u^* \in C_u\right]. \tag{1}$$

Events without an existing target (new facts, small talk) never enter the denominator.

An exact factorisation. AURA applies an update, retire or forget operation only if the memory identifier it names was in C_u; any other identifier is discarded. Let R_u be the event $m_u^* \in C_u$, O_u the event that the extractor returns the correct operation on m_u^*, A_u the event that the user accepts it, and S_u the event that the change takes effect. Because S_u requires O_u, and O_u can only take effect if R_u holds,

$$P(S) = P(R)\,P(O \mid R)\,P(A \mid O, R). \tag{2}$$

The gate acts on the last factor (it blocks wrong operations and may lose correct ones through user error); it cannot change $P(R)$. This is the formal version of the paper's title. (The factorisation concerns changes proposed by the extractor; AURA's optional "is this still true?" checks, which ask the user directly, are a separate path and were not used in any experiment.) When R_u fails, the extractor often stores the new value as a fresh memory, so the outdated record stays active beside it: a missed update usually appears as a conflict, not as silence.

Reachability versus retrieval recall. If there were no minimum score and no recent-memory slots, CR would equal Recall@k of the target on change events. AURA's selection differs in both respects: the minimum score can remove a target that ranks within the top k, and the recent slots can add a target that search did not find. We therefore report, next to CR, the share of targets within the top k of the unfiltered ranking (Sec. IX). CR is not a new kind of retrieval metric; it is recall measured where it matters for updating, on the final prompt and only on events that require a change.

Quantities reported. We keep four quantities apart: reachability (is m_u^* in C_u?), retrieval quality (how search ranks the target), update correctness (given R_u, is the right change proposed and accepted?), and final memory quality (after many turns, are facts current, supported and preserved?). The simulations and real runs measure the last; Sections VIII and IX measure the first two; the real runs also report missing targets per change turn.

IV. System: User-Controlled Memory in AURA

A. Memory store and operations

AURA keeps memories in SQLite. A memory has text, a category (preference, personal, people, work, plan, other), a status (`active`, `superseded` or `outdated`), a pointer to the memory that replaced it, a source and its conversation, and timestamps. After each user message a Gemini model (default `gemini-3.5-flash-lite`) receives the message and a list of candidate memories and returns operations: *add*, *update* (replace a value), *retire* (the fact no longer holds), or *forget*. An update keeps the old row as `superseded` with a link to its successor; retire marks the old row `outdated`. The extractor can refer only to memory identifiers that appear in its prompt.

TABLE I
CLOSEST WORK: WHAT DECIDES A MEMORY CHANGE, HOW STORED MEMORY REACHES THE COMPONENT THAT CHANGES IT, AND HOW THIS DIFFERS FROM THE PRESENT STUDY.

Work	Who decides a change	How stored memory reaches it	Evaluation	Difference from this study
Mem0 [3]	LLM chooses add / update / delete / no-op	most similar stored memories per new fact	LoCoMo	same pipeline shape; no user gate; target visibility not measured
TrustMem [4]	trained policy (write / revise / prune)	current memory with the new chunk; verifier sees touched or retrieved entries	MemoryAgentBench, HaluMem, Mem-α	omission = new information not kept; trained; no user gate
AgeMem [5]	trained policy with memory tools	retrieval is one of the policy's tools	five long-horizon benchmarks	visibility of the update target learned, not measured
Supersede [6]	agent rewrites bounded notes	whole notes field, no retrieval	LongMemEval KU ($n = 78$)	maintenance under a memory budget, not visibility
PAMU [12]	averaging and change detection	preference vector	LoCoMo	preferences, not stored fact records
REALM [7]	reconsolidation from retrieval feedback	graph retrieval for answering	LoCoMo, LongMemEval	retrieval reorganises memory; no update-target selection
MemGate [8]	— (read-time admission)	learned gate over retrieved memories	leakage, jailbreak, LoCoMo	controls what reaches answers; cannot raise reachability
This paper	extractor proposes, user approves	search matches plus recent memories	simulation, two Gemini models, LongMemEval KU ($n = 72$)	measures and varies whether the target reaches the updater

TABLE II
MEMORY AND PROPOSAL RECORDS.

Field	Meaning
`content`	one third-person sentence, with date context for time-bound facts
`category`	preference, personal, people, work, plan, other
`status`	`active`, `superseded` (replaced), `outdated` (no longer true)
`superseded_by`	identifier of the replacing memory
`source`, conversation	where the fact came from
proposal: action	`update`, `retire` or `forget`
proposal: status	`pending`, `accepted`, `rejected` (with decision time)

B. Data model

Table II lists the fields that matter for the evaluation. Nothing is overwritten in place by an update: the old row stays with status `superseded` and points to its successor, so every change can be audited and, if needed, reversed by the user.

C. Extraction prompt

After each batch of new user messages the extractor receives the date, the candidate memories as lines of the form `[id] (category) text`, optional earlier context and the new messages. The instructions tell it to store only what the *user* said or confirmed (preferences, personal details, people, work, plans, routines), not questions, general knowledge, assistant suggestions or one-off requests, to write one short third-person sentence per fact, and to choose among the four operations. The output is constrained to a JSON schema with the fields action, memory_id, content and category. An operation that names an identifier that was not in the candidate list is ignored by the application. This is the point at which candidate selection limits what can change.

D. The review gate

When "ask before changing memories" is on (the default), *add* operations are applied directly and every operation that touches an existing memory becomes a stored *proposal* (action, target memory, proposed text, conversation). The user accepts or rejects it; nothing changes until then. We call this policy **gate**. The comparison policies are **append** (only additions; a changed fact is stored as a second fact), **auto** (all operations applied at once), and **gate all** (also new facts wait for approval; emulated in the harness, not an AURA setting).

E. Candidate selection

The prompt contains candidate memories chosen from the store. By default these are the 12 best search matches for the user's message plus the 8 most recently created memories. Search combines the cosine similarity of MiniLM sentence embeddings [14] (`all-MiniLM-L6-v2`, run locally) and a lexical overlap score, $0.85 \cdot \cos + 0.15 \cdot \text{lex}$, and drops matches below a minimum score of 0.25. We evaluate four modes, selected by an environment variable:

- **default**: as above;
- **no minimum**: the same, without the 0.25 cut-off;
- **sentence**: the message is split into sentences and each is searched on its own, so that an unrelated sentence does not dilute the query;
- **adaptive** (F2): sentence search without minimum score, and a larger number of matches (up to 24) when the message contains a change cue (words such as "moved", "stopped", "no longer", "forget").

Using F2 enlarges the prompt (Sec. XI); the minimum score exists to keep unrelated memories out (Fig. 3). Because F2 bundles three changes, Section IX separates them and compares F2 with plain larger lists of the same size.

F. Re-proposal (F1)

A rejected proposal normally ends the matter. With F1 a rejected change may be proposed again, at most N times, but only after the user has said something new about it (we use $N = 2$). The aim is to recover from a rejection that was a slip without nagging the user. Concretely, when the extractor repeats a rejected change, the application compares the time

Algorithm 1: Candidate selection (modes *default* and *adaptive*)

Input: user text x, mode, store, minimum score $\tau = 0.25$
Output: candidate memories shown to the extractor

$f \leftarrow 0$ if mode is no-minimum or adaptive, else τ;
if *mode is sentence or adaptive* **then**
  $k \leftarrow 24$ if mode is adaptive and x contains a change cue, else 12;
  foreach $q \in \{x\} \cup \text{sentences}(x)$ **do**
    foreach *memory* m *in* $\text{search}(q, k, f)$ **do**
      keep the best score per memory;
  $R \leftarrow$ the k memories with the best scores;
else
  $R \leftarrow \text{search}(x, 12, f)$;
return $R \cup$ the 8 most recent memories;

TABLE III
WHAT IS REAL AND WHAT IS SIMULATED.

Part	Sim. runs	Real runs
Store, operations, proposals, search	AURA code	AURA code
Sentence embeddings	MiniLM	MiniLM
Extractor	simulated, rate e	Gemini
Reviewing user	simulated, slip u	simulated, $u = 0$
Conversations	scripted personas	same personas

of the last user message with the time of the last rejection and creates a new proposal only if the user has written since and the number of earlier rejections is at most N. An identical proposal that is still pending is never duplicated, and a newer proposal for the same memory replaces an older pending one.

V. Evaluation Method

All experiments run AURA's real server code, real SQLite schema and real local embedding model on a throw-away database. What is real and what is simulated is listed in Table III.

A. Personas and ground truth

A persona has 8 of 12 fact types (city, employer, dog, sister, diet, gym time, language, car, project, coffee, hobby, birthday) and five sessions of about 23 user turns. Facts are stated, later updated ("I've just moved to Hyderabad"), retired ("I stopped going to the gym"), forgotten on request, repeated, and mixed with distractors that must not become memories (hypotheticals, questions, assistant suggestions, facts about friends). A store starts with 30 unrelated true background facts and gains 10 between sessions, imitating earlier use. The true state is known after every turn. Table IV lists one persona in full; it is used again in the worked example (Sec. XI-B).

In the ablation runs a restating turn ("Like I said, ...") follows each update, which is what F1 needs to act on.

B. Simulated extractor and user

The simulated extractor returns what a correct model would return for the turn, using only the memories shown in the prompt; a change whose target is not shown becomes an add, as a real model would do. Errors are injected at rate e: a correct operation is dropped, a distractor becomes a false memory, or a true memory shown to the model is retired or overwritten with a false value. All policies face the same random draws. The simulated user reviews proposals at the end of a session, accepts correct ones and rejects wrong ones, and errs with probability u (default 0.05).

TABLE IV
PERSONA 1 OF THE REAL-MODEL RUNS: WHAT THE USER SAYS IN EACH SESSION, AND THE TRUE STATE AT THE END.

- Session 1: 'I go to the gym at 7 am every day.' 'My birthday is on 21 September.' 'I take my coffee as a cappuccino.' 'My dog is called Mithu.'
- Session 2: 'Like I said, my dog is called Mithu.' 'My friend Rohan just bought a Thar, it looks great.' 'I live in Kolkata.' 'I drive a Seltos.' 'I work at Swiggy.' 'I'm leading a project called Lotus at work.'
- Session 3: 'Like I said, i go to the gym at 7 am every day.' 'What's the weather like in Shimla today?' 'I moved my gym time to 6 pm.' 'My project wrapped up last week, I'm between projects.'
- Session 4: 'Like I said, i work at Swiggy.' 'Please forget what I told you about my birthday.' 'Can you set a timer for ten minutes?' 'I've just moved to Pune.'
- Session 5: 'Is it true that octopuses have three hearts?' 'Like I said, i work at Swiggy.' 'I've just moved to Chennai.' 'I sold my old car and bought a Swift.'
- Truth at the end: gym: 6 pm; birthday: no longer true; coffee: as a cappuccino; pet: Mithu; city: Chennai; car: Swift; employer: Swiggy; project: no longer true. Asked to forget: birthday (21 September).

C. Scoring rules

Memories written by a model are scored by the *values* they mention (for example "Pune", "Swift", "6 am"), using the persona's slots and a list of alternative wordings. A memory mentioning a value that is true at the end is *true*. One that mentions a value the user gave earlier and that is no longer true is *stale*, unless it records that the fact ended ("finished the project and is between projects", after a retire request), which is *ended* and counted as correct. A memory about a distractor value is *hypothetical* when it is worded as a hypothetical, *third-party* when it concerns another person (allowed), and *false* otherwise. A memory that mentions no known value is *other*. The exact lists are given in Appendix A.

D. Metrics

F1 compares the persona's active memories with the true facts. **Stale** counts active memories holding a value the user gave earlier that is no longer true. **False** counts active memories the user never stated about themselves. **Destroyed** counts true memories, persona and background, that ended up superseded, outdated or deleted by a wrong change; F1 does not see the background memories, so this is the safety metric. **Prompts/100** is the number of review cards per 100 user turns. **Reachability** is the share of change requests whose target memory is among the extractor's candidates. **Prompt tokens** are summed over a persona run. With T the set of persona facts that are true at the end, A the set of active persona memories scored true or

ended, and P the set of all active persona memories, precision is $|A|/|P|$, recall is the share of T covered, and F1 their harmonic mean. **Exact state** is 1 for a persona whose final memory has every true fact, no stale and no false fact, and 0 otherwise.

E. Real-model runs and scoring

Real runs use Gemini as the extractor for 12 personas, one seed, with the simulated user as an oracle ($u = 0$), for the policies auto and gate and four variants (baseline, F1, F2, F1+F2). They were repeated for baseline and F1+F2 with a second model (`gemini-3.1-flash-lite`). Because models word memories freely, a memory is scored by value: the automatic scorer maps each active memory to a persona fact and a label (true, stale, ended, hypothetical, third party, other). The author hand-labelled every memory of the baseline and F1+F2 runs (195 and 175 memories). There were 3324 extraction calls and 0 failed. Requests were rate-limited (12 per minute), each persona was played in its own isolated database, and the extractor was the only part of the system that called the model; the reviewing user was the oracle described above and judged a proposal by the same value-based rules as the scorer. This has a consequence that we discuss with an example (Sec. XI-B): the oracle can reject a correct proposal whose wording the rules do not recognise.

F. Statistics

Differences between variants are paired by persona and seed. We report the mean difference with a 95% bootstrap interval (4000 resamples, fixed seed) and an exact two-sided sign test on the non-tied pairs. No correction for multiple tests is applied; the tests run are 210 (simulated ablation) and 48 (real runs), so isolated p-values near 0.03 should not be read as findings. Reachability is reported with Wilson intervals [15], and agreement between hand and automatic labels with Cohen's κ [16]; since there is a single annotator this measures the scorer against the annotator, not human–human agreement.

VI. Results: What the Gate Protects

Fig. 1 and Table V compare the four policies over extractor error rates e (60 persona runs per cell: 20 personas, 3 seeds).

Wrong writes. Under auto-apply the extractor's errors destroy true facts: 0.78, 1.60, 2.83 and 4.60 per persona at $e = 0.05, 0.1, 0.2, 0.3$. The gate reduces this to 0.05, 0.03, 0.18 and 0.25, i.e. by 94–98%. The remainder comes from the simulated user's slips. The price is 20–33 review prompts per 100 turns; requiring approval for new facts as well removes false new facts but needs about 2.7 times as many prompts as the gate.

Missed updates. The gate does not help here. At $e = 0$ the append-only policy leaves 5.00 stale facts per persona by construction, and the gate still leaves 1.80, slightly more than auto-apply (1.57); across the error sweep the gate leaves between 1.8 and 2.2 stale facts, auto-apply between 1.6 and 1.9. F1 of the gate is below auto-apply at $e = 0$ (0.854 vs. 0.881) and about equal at higher e. The gate thus buys safety at a small cost in F1, and does nothing for stale facts.

Earlier evidence for the candidate bottleneck. An earlier run of the same simulation (without restating turns) already pointed at candidate selection: removing the 0.25 minimum score from the extractor's candidate search, and nothing else, roughly halved the stale facts left at low error rates under both auto-apply and the gate (Fig. 2). That a change in how candidates are chosen moves the stale count far more than the choice between gate and auto-apply is what prompted the reachability analysis below.

User slips. Table VI varies the simulated user's slip rate at $e = 0.1$. F1 falls from 0.808 to 0.732 and stale facts rise from 1.75 to 2.52 as u goes from 0 to 0.3: a gate is only as good as the review.



Fig. 1. Policies under extractor errors (simulated, shaded: 95% CI). (a) F1. (b) True facts destroyed by wrong changes. (c) Review prompts per 100 turns.

TABLE V
REVIEW POLICIES VERSUS EXTRACTOR ERROR RATE e (SIMULATED, USER SLIP $u = 0.05$, MEAN PER PERSONA).

e	policy	F1	stale	false	destroyed	prompts/100
0	append-only	0.597 ± 0.025	5.00	0.00	0.00	0.0
	auto-apply	0.881 ± 0.018	1.57	0.00	0.00	0.0
	gate changes	0.854 ± 0.022	1.80	0.00	0.00	19.5
	gate all	0.861 ± 0.023	1.62	0.00	0.00	65.8
0.1	append-only	0.581 ± 0.023	4.58	0.33	0.00	0.0
	auto-apply	0.798 ± 0.026	1.68	0.43	1.60	0.0
	gate changes	0.797 ± 0.026	1.85	0.33	0.03	24.4
	gate all	0.795 ± 0.029	1.83	0.02	0.05	69.0
0.3	append-only	0.523 ± 0.034	3.87	1.20	0.00	0.0
	auto-apply	0.650 ± 0.032	1.92	1.33	4.60	0.0
	gate changes	0.653 ± 0.033	2.18	1.22	0.25	33.4
	gate all	0.707 ± 0.030	1.98	0.07	0.40	77.6

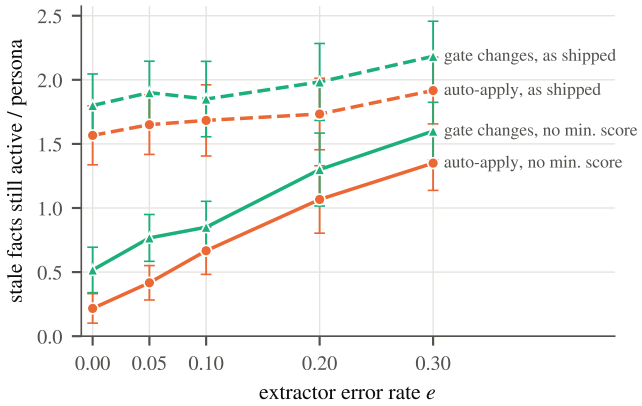


Fig. 2. Stale facts left active, as shipped versus candidate search without the minimum score (earlier simulation run, no restating turns).

TABLE VI
GATE UNDER USER SLIPS (SIMULATED, $e = 0.1$).

slip u	F1	stale	wrong changes applied	prompts/100
0	0.808 ± 0.026	1.75	0.02	24.5
0.1	0.783 ± 0.028	2.00	0.28	24.4
0.2	0.757 ± 0.027	2.33	0.42	24.0
0.3	0.732 ± 0.029	2.52	0.62	24.1

VII. Results: Does Memory Search Find the Right Memory?

Candidate selection builds on memory search, so we first measured search alone. Each store holds one persona's current facts plus $N \in \{25, 100, 350\}$ unrelated facts; every stored fact is asked about with two natural questions ("Which company do I work for?"), 2178 queries in all over 20 personas and 3 seeds. A query is a hit when the right memory is among the top six results. We compare AURA's hybrid score with the 0.25 minimum, the same score without a minimum, embeddings only and keywords only (the fallback used when the embedding model is unavailable). A check that re-scoring reproduces AURA's real top-six lists succeeded for all queries (differences only from ties).

Table VII shows that AURA's search finds the right fact for only 0.700 of the questions at $N = 350$ and returns nothing at all for 0.281, while the same score without a minimum reaches 0.891. The cause is visible in the scores: for 0.293 of the questions the right fact scores below 0.25 on the hybrid score, because a natural question about a fact ("What do I do for fun?") shares few words and only moderate embedding similarity with the stored sentence ("The user's main hobby is chess").

The failure is concentrated in a few kinds of fact (Table VIII). Facts with a distinctive topic word (dog, sister, birthday, coffee, language) are found almost always; facts that people phrase in many ways or that name no topic word (city, employer, diet, car) are often not found with the 0.25 minimum, although the no-minimum search finds most of them.

The minimum score is there for a reason. Table IX and Fig. 3 show the trade-off: lowering it raises the share of stored facts found but also the share of questions about things the user never said that still return memories (noise), up to 1.00 with no minimum. A single threshold therefore has to compromise between recall and silence, and at 0.25 AURA returns something for about half of the never-stated questions at $N = 350$ while finding only 0.70 of the stored ones. For a conversational recall tool this may be tolerable; for the extractor, whose job is to see the memory it must change, it is a bottleneck.

VIII. Results: What the Extractor Can See

For each of 30 stores per size we generated update, retire and forget requests (2610 in all) and checked whether the memory the request refers to is among the extractor's candidates. The store holds N unrelated memories besides the ones the persona has stated.

Reachability falls from 0.898 with an empty store to 0.690 at $N = 350$ (Fig. 4, Table X). Even with an empty store about one change in ten is out of reach because the 0.25 minimum drops weak matches. Without the minimum the figure is 0.911 at $N = 350$. The recent-memory slots help only at small N.

Table XI compares the modes, with the mean number of candidates in parentheses. When the change shares the message with two unrelated sentences, which is common in natural conversation, default selection reaches only 0.286 of changes at $N = 350$, because the embedding of the whole message is

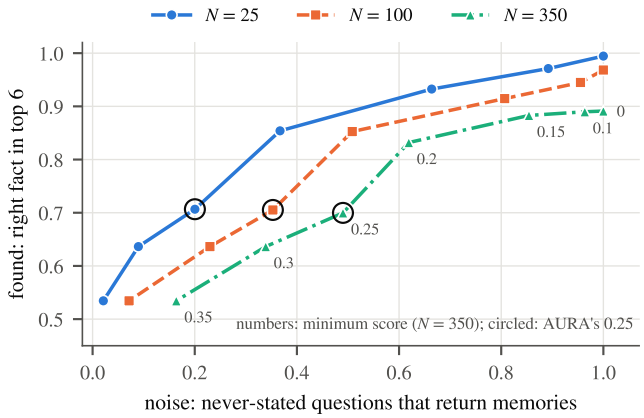


Fig. 3. Retrieval trade-off of the minimum score: share of stored facts found in the top six versus share of never-stated questions that still return memories. Circled: AURA's 0.25.

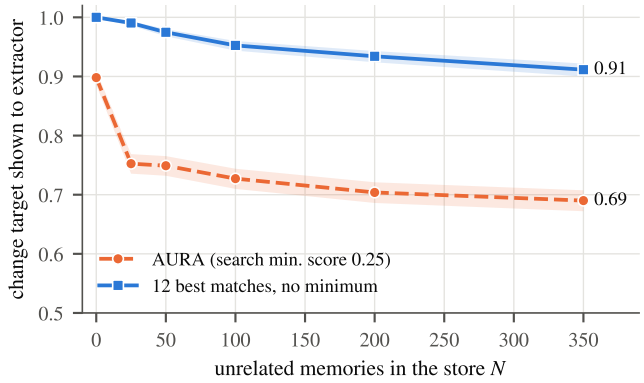


Fig. 4. Share of change requests whose target memory is shown to the extractor, versus store size N (shaded: Wilson 95% CI).

dominated by the unrelated content. Removing the minimum score gives 0.607; per-sentence search alone 0.690; and the adaptive mode 0.866, at 28.1 candidates on average instead of 8.3. Per-sentence search makes a mixed message behave like a single sentence. The minimum score is not free to remove, because it also suppresses unrelated matches (Table IX), and a larger prompt costs tokens.

By kind of change. Table XII splits the result at $N = 350$ by update, retire and forget requests. Retire and forget requests are reached more often than updates by the default selection in single-sentence turns, but all three suffer in mixed turns, and the adaptive mode restores them to a similar level. (Single-sentence turns have one sentence, so the sentence mode equals the default there by construction.)

By kind of fact. The aggregate hides large differences (Table XIII, Fig. 5). For single-sentence changes at $N = 350$ the default selection reaches the car in only 0.04 of cases, the employer in 0.18 and the city in 0.34; the adaptive mode raises these to 0.84, 0.95 and 0.49. The car case is the one in our worked example. The city remains the weakest even with F2: a statement such as "We relocated to Pune last month" contains no word that points to the old city, and with 350 other

TABLE VII
RETRIEVAL: SHARE OF QUESTIONS WHOSE ANSWER MEMORY IS IN THE TOP SIX.

N	search variant	hit@6 [95% CI]	top-1	no result
25	AURA (hybrid, min 0.25)	0.707 [0.672, 0.739]	0.705	0.287
	hybrid, no minimum	0.994 [0.986, 0.998]	0.920	0.000
	embedding only, no minimum	0.994 [0.986, 0.998]	0.897	0.000
	keywords only (fallback)	0.606 [0.570, 0.641]	0.581	0.335
100	AURA (hybrid, min 0.25)	0.705 [0.671, 0.737]	0.690	0.281
	hybrid, no minimum	0.968 [0.953, 0.979]	0.861	0.000
	embedding only, no minimum	0.957 [0.940, 0.970]	0.828	0.000
	keywords only (fallback)	0.606 [0.570, 0.641]	0.576	0.325
350	AURA (hybrid, min 0.25)	0.700 [0.665, 0.732]	0.687	0.281
	hybrid, no minimum	0.891 [0.866, 0.912]	0.833	0.000
	embedding only, no minimum	0.872 [0.846, 0.894]	0.787	0.000
	keywords only (fallback)	0.606 [0.570, 0.641]	0.567	0.309

TABLE VIII
RETRIEVAL BY KIND OF FACT AT $N = 350$ (HIT@6; MEAN COSINE SIMILARITY OF THE RIGHT FACT).

fact	queries	AURA hit@6	no-min. hit@6	mean cosine
city	60	0.27	0.37	0.253
employer	76	0.24	0.89	0.248
dog	68	1.00	1.00	0.595
sister	44	1.00	1.00	0.582
diet	56	0.09	0.41	0.175
gym time	68	1.00	1.00	0.480
language	66	1.00	1.00	0.494
car	64	0.34	1.00	0.236
project	62	1.00	1.00	0.436
coffee	56	1.00	1.00	0.483
hobby	66	0.65	1.00	0.400
birthday	40	1.00	1.00	0.602

memories the right one is often not among the 24 best matches. F2 is therefore an improvement, not a complete solution, and a store with many place-like facts will need better candidate selection than ours.

IX. RESULTS: WHICH PART OF CANDIDATE SELECTION MATTERS

The adaptive mode changes three things at once: it searches each sentence, it drops the minimum score, and it shows up to 24 instead of 12 matches when the message contains a change cue. It also shows more candidates, so a fair question is whether a plain larger list would do as well. We answer this on a second, independently generated test set.

Test set. 40 personas, each with 12 facts (city, employer, dog's name, sister's city, diet, gym time, language being learned, car, project, coffee, hobby, birthday) whose values and new values are drawn per persona. Each persona has 350 unrelated memories, 30% of them *similar* (the same relation about another person, such as "The user's friend Rohan lives in Delhi") and the rest generic; no unrelated memory mentions any of the persona's values. Stores of $N \in \{0, 25, 50, 100, 200, 350\}$ unrelated memories are built incrementally, with the persona's facts older than every unrelated memory. Each persona has 80 change events (44 updates, 28 retires, 8 forgets). Every change is written in a *cue* form, which contains a word from AURA's change-cue list, and a *cue-free* form, which contains none (checked automatically); each form appears as a single sentence and as a mixed turn with two unrelated sentences. This gives 3,200 events per store size and 19,200 event–store pairs. The cue-free half is held out from the design of the cue list and tests whether the adaptive mode depends on wording.

Conditions. Table XIV lists 15 selection conditions: the four modes in AURA, four ablations that switch the adaptive mode's parts on one at a time or in pairs, three *same-budget* baselines that always show 24 matches, and four retrieval baselines added after the pre-registered design: BM25 keyword ranking ($k_1 = 1.2$, $b = 0.75$, AURA's tokeniser, only memories sharing a term with the message) with 12 and 24 matches, and reciprocal-rank fusion of BM25 with AURA's ranking, for the whole message and per sentence. All conditions are computed from AURA's own search; for the four modes that exist in AURA, a test confirms that the computed candidate lists are identical to those of AURA's selection code. The server code was not changed.

Statistics. The persona is the unit (40). Rates come with persona-clustered bootstrap intervals (10,000 resamples). Six comparisons were fixed before the run and are Holm-corrected; they use exact sign tests over personas on paired per-persona rates (Table XV). The four comparisons for the added retrieval baselines form a second, separately Holm-corrected family. Other comparisons are labelled exploratory.

The default selection. With 350 unrelated memories the default reaches 0.667 of targets [0.654, 0.680], 0.774 for single sentences and 0.559 for mixed turns. It already loses a third of targets at $N = 25$ (0.682) and then stays flat (Fig. 6a): the minimum score, not the size of the store, is the main limit. At $N = 350$, 0.110 of targets rank within the top 12 but score below 0.25.

Which parts matter. Removing the minimum score alone adds +0.110 and per-sentence search alone adds +0.100 (both 40 of 40 personas, Holm $p < 0.001$). Showing 24 matches on a change cue, with the minimum score kept, adds only +0.007: the minimum score leaves most of the extra places empty (the default with $k = 24$ shows 14.3 candidates on average and reaches 0.680). The adaptive mode, which combines all three, gains +0.178 over the default.

TABLE IX
MINIMUM SCORE TRADE-OFF: FOUND / NOISE FOR N AND MINIMUM SCORE. FOUND: SHARE OF STORED-FACT QUESTIONS ANSWERED; NOISE: SHARE OF NEVER-STATED QUESTIONS THAT STILL RETURN MEMORIES.

N	0	0.1	0.15	0.2	0.25	0.3	0.35
25	0.99/1.00	0.97/0.89	0.93/0.66	0.85/0.37	0.71/0.20	0.64/0.09	0.53/0.02
100	0.97/1.00	0.94/0.96	0.91/0.81	0.85/0.51	0.71/0.35	0.64/0.23	0.53/0.07
350	0.89/1.00	0.89/0.96	0.88/0.85	0.83/0.62	0.70/0.49	0.64/0.34	0.53/0.16

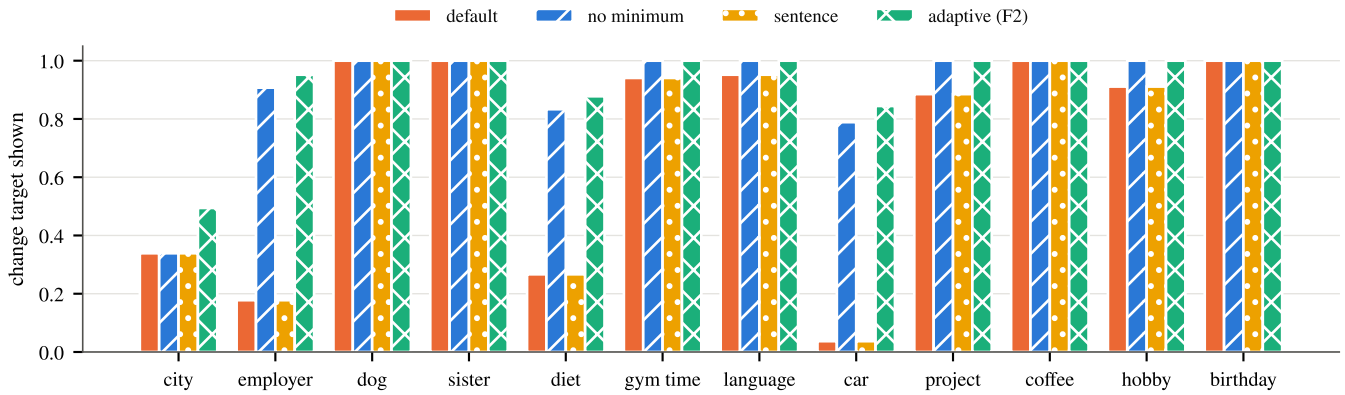


Fig. 5. Share of change requests whose target memory is shown to the extractor, by kind of fact ($N = 350$, single-sentence changes, 4 selection modes).

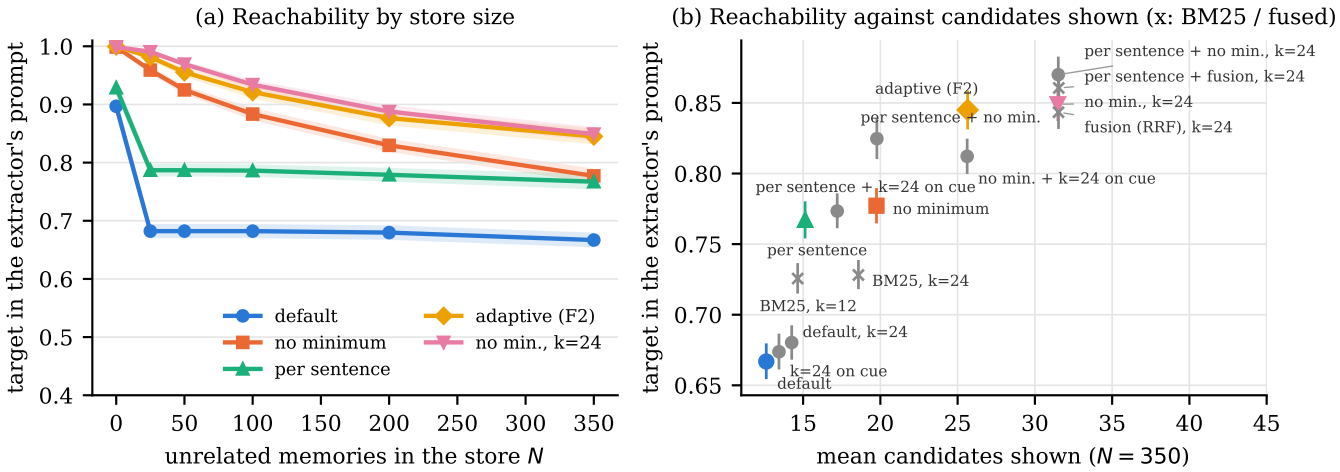


Fig. 6. Selection study. (a) Reachability against store size for five conditions (shaded: persona-clustered 95% CI). (b) Reachability against the mean number of candidates shown at $N = 350$, all eleven conditions (bars: 95% CI).

A same-budget baseline matches it. A plain list of the 24 best whole-message matches without a minimum score reaches 0.849, against 0.845 for the adaptive mode (difference -0.004 [-0.013, +0.004], 15 personas favouring the adaptive mode and 17 the plain list, Holm $p = 0.860$). The adaptive mode reaches this level while showing 19% fewer candidates (25.6 against 31.5), because it enlarges the list only on a cue. Its possible advantage over the plain list is per-sentence search on mixed turns (exploratory: +0.016, $p = 0.061$, not significant); its clear disadvantage is the dependence on cue words: on cue-free messages it falls back to 12 matches and reaches -0.032 less than the plain list (exploratory, $p = 0.003$), and within the adaptive mode cue-free messages are reached -0.080 less often than cue messages (Holm $p < 0.001$). Combining per-sentence search, no minimum score and 24 matches always gives the highest value, 0.870 (exploratory: +0.021 over the plain list).

Keyword and fused rankings. BM25 on its own reaches 0.726 with up to 12 matches, more than the default (+0.059, Holm $p < 0.001$), because it has no minimum score; with up to 24 matches it reaches 0.728, since often fewer than 24 memories share a word with the message (18.6 candidates on average), and it stays well below the same-size embedding list (-0.121). Fusing BM25 with AURA's ranking reaches 0.843, the same as the plain embedding list (-0.006, Holm $p = 0.377$),

TABLE X
REACHABILITY FOR SINGLE-SENTENCE CHANGES (95% WILSON INTERVAL).

N	shown (default)	shown (no minimum score)
0	0.898 [0.886, 0.909]	1.000 [0.999, 1.000]
25	0.752 [0.736, 0.769]	0.990 [0.986, 0.994]
50	0.749 [0.732, 0.765]	0.975 [0.968, 0.980]
100	0.727 [0.710, 0.744]	0.952 [0.944, 0.960]
200	0.704 [0.686, 0.721]	0.934 [0.924, 0.943]
350	0.690 [0.672, 0.707]	0.911 [0.900, 0.922]

TABLE XI
REACHABILITY BY SELECTION MODE (MEAN NUMBER OF CANDIDATES IN PARENTHESES).

N	turn type	default	no min.	sentence	adaptive
0	single sentence	0.898 (8.2)	1.000 (11.4)	0.898 (8.2)	1.000 (11.4)
	mixed turn	0.747 (8.1)	0.998 (11.4)	0.898 (8.2)	1.000 (11.9)
350	single sentence	0.690 (11.3)	0.911 (19.8)	0.690 (11.3)	0.936 (28.2)
	mixed turn	0.286 (8.3)	0.607 (19.8)	0.690 (11.3)	0.866 (28.1)

and per-sentence fusion 0.861 (-0.009 against per-sentence embedding search, Holm $p = 0.220$). On this test set, adding keyword evidence does not improve on a larger embedding list. A first BM25 run that kept memories with zero score ranked them in insertion order, which placed the persona's own (oldest) facts first and inflated BM25 strongly; we report only the corrected run and mention this because the artefact is easy to reproduce by accident.

A cross-encoder reranker. As a stronger retrieval baseline we reranked the 50 best whole-message matches of AURA's search (no minimum score) with a cross-encoder (`ms-marco-MiniLM-L-6-v2`, quantised, run locally) and showed the top 24 plus the recent memories, on the same events and stores at $N = 350$. The reranker reaches 0.887 [0.876, 0.898] with 31.5 candidates, the same number as the plain list: +0.038 over the plain list (36 personas better, 1 worse, Holm $p < 0.001$), +0.042 over the adaptive mode, and +0.017 over per-sentence search with 24 matches (Holm $p = 0.050$, not significant). Its ceiling is the pool: the target is among the 50 candidates in 0.903 of events. It is weakest on mixed turns (0.834), where per-sentence search helps the other selections; reranking per sentence was not tried. The five reranker comparisons form a third Holm family, added after the pre-registered design; the reranker was run on the authors' machine because its download is blocked in the environment used for the other runs (Table XIV, last row; Fig. 6 does not include it).

What no variant fixes. At $N = 350$, 10.0% of change events are reached by none of the 15 conditions (7.7% also not by the reranker), and only 0.849 of targets are among the 24 best matches for the whole message even without a minimum score. These are messages that name the new value but share little with the stored sentence, for example a car model mentioned without the word "car", or a city statement facing many similar place facts about other people: the car fact is reached in 0.069 of events by the default and 0.500 by the adaptive mode. Similarity-based candidate lists of this size have a ceiling, and closing the rest needs a different mechanism (Sec. XIII).

TABLE XII
REACHABILITY BY KIND OF CHANGE AT $N = 350$.

turn type	change	default	no min.	sentence	adaptive
single sentence	update	0.734	0.907	0.734	0.943
	retire	0.697	0.989	0.697	0.995
	forget	0.649	0.870	0.649	0.894
mixed turn	update	0.314	0.610	0.734	0.894
	retire	0.241	0.567	0.697	0.906
	forget	0.288	0.628	0.649	0.819

TABLE XIII
REACHABILITY BY KIND OF FACT AT $N = 350$ (SINGLE-SENTENCE CHANGES).

fact	changes	default	no min.	sentence	adaptive
city	180	0.34	0.34	0.34	0.49
employer	270	0.18	0.91	0.18	0.95
dog	270	1.00	1.00	1.00	1.00
sister	90	1.00	1.00	1.00	1.00
diet	180	0.27	0.83	0.27	0.88
gym time	270	0.94	1.00	0.94	1.00
language	270	0.95	1.00	0.95	1.00
car	270	0.04	0.79	0.04	0.84
project	270	0.89	1.00	0.89	1.00
coffee	270	1.00	1.00	1.00	1.00
hobby	180	0.91	1.00	0.91	1.00
birthday	90	1.00	1.00	1.00	1.00

Relation to the first test set. The direction of every effect agrees with Sec. VIII, but the levels differ between the two test sets (for example, mixed turns are reached 0.286 of the time there and 0.559 here under the default), because the phrasing of changes and of the surrounding sentences differs. Reachability levels are therefore properties of a system *and* a conversation distribution; comparisons between selections on the same events are the robust result.

A. Real-model pilot: oracle target and same-size list

The selection study measures reachability only. To see whether reachability turns into correct updates with a real extractor, we ran a pilot with Gemini (`gemini-3.5-flash-lite`), the review gate and an oracle reviewer, on the personas of the new test set: 18 turns per persona (12 change turns, the others new facts and unrelated chatter), 100 unrelated memories, four selections. Besides the default, the adaptive selection and a plain list of 24 matches without minimum score, an *oracle-target* condition adds the true target to the default candidates; it is a diagnostic upper bound on what better selection could give, not a method. Eight personas were planned. Two (p07 and p08) could not be completed: the API quota was exhausted (HTTP 429), first in seven of their eight runs and again in a repeat attempt, which finished only two of those seven; we excluded both personas, and report the six personas whose four runs completed without any error (72 change turns per selection). The result is a pilot: six personas, one seed, one model, and persona-clustered intervals that are wide.

TABLE XIV
SELECTION STUDY AT $N = 350$: FINAL-PROMPT REACHABILITY (PERSONA-CLUSTERED 95% CI), MEAN NUMBER OF CANDIDATES SHOWN, AND REACHABILITY FOR SINGLE-SENTENCE, MIXED AND CUE-FREE MESSAGES. SENT.: PER-SENTENCE SEARCH; k: MATCHES SHOWN (WITHOUT / WITH A CHANGE CUE). 3,200 CHANGE EVENTS, 40 PERSONAS.

condition	role	sent.	floor	k	reach. [95% CI]	cand.	single / mixed / cue-free
default	in AURA	no	0.25	12	0.667 [0.654, 0.680]	12.6	0.774 / 0.559 / 0.647
no minimum	in AURA	no	–	12	0.777 [0.765, 0.790]	19.7	0.866 / 0.688 / 0.764
per sentence	in AURA	yes	0.25	12	0.767 [0.754, 0.780]	15.1	0.774 / 0.760 / 0.741
adaptive (F2)	in AURA	yes	–	12/24	0.845 [0.831, 0.859]	25.6	0.889 / 0.801 / 0.805
per sentence + no min.	ablation	yes	–	12	0.825 [0.810, 0.839]	19.8	0.866 / 0.783 / 0.805
k=24 on cue	ablation	no	0.25	12/24	0.674 [0.661, 0.687]	13.4	0.779 / 0.569 / 0.647
no min. + k=24 on cue	ablation	no	–	12/24	0.812 [0.800, 0.825]	25.6	0.889 / 0.736 / 0.764
per sentence + k=24 on cue	ablation	yes	0.25	12/24	0.773 [0.761, 0.786]	17.2	0.779 / 0.768 / 0.741
default, k=24	budget	no	0.25	24	0.680 [0.668, 0.693]	14.3	0.784 / 0.576 / 0.660
no min., k=24	budget	no	–	24	0.849 [0.837, 0.861]	31.5	0.913 / 0.786 / 0.838
per sentence + no min., k=24	budget	yes	–	24	0.870 [0.857, 0.883]	31.5	0.913 / 0.828 / 0.855
BM25, k=12	lexical	no	–	12	0.726 [0.715, 0.737]	14.6	0.724 / 0.727 / 0.686
BM25, k=24	lexical	no	–	24	0.728 [0.718, 0.739]	18.6	0.724 / 0.732 / 0.688
fusion (RRF), k=24	fused	no	–	24	0.843 [0.832, 0.856]	31.5	0.899 / 0.788 / 0.816
per sentence + fusion, k=24	fused	yes	–	24	0.861 [0.848, 0.874]	31.5	0.899 / 0.823 / 0.835
cross-encoder rerank of top 50, k=24	reranker	no	–	24	0.887 [0.876, 0.898]	31.5	0.939 / 0.834 / 0.878

TABLE XV
PRE-REGISTERED COMPARISONS AT $N = 350$ (PAIRED PER-PERSONA DIFFERENCES IN REACHABILITY, HOLM-CORRECTED SIGN TESTS).

comparison ($N = 350$)	difference [95% CI]	personas $+/-/=$	Holm p
adaptive − default	+0.178 [+0.166, +0.190]	40/0/0	<0.001
adaptive − no min., k=24	-0.004 [-0.013, +0.004]	15/17/8	0.860
per sentence − default	+0.100 [+0.090, +0.110]	40/0/0	<0.001
no minimum − default	+0.110 [+0.101, +0.120]	40/0/0	<0.001
k=24 on cue − default	+0.007 [+0.004, +0.010]	15/0/25	<0.001
adaptive: cue-free − cue messages	-0.080 [-0.102, -0.057]	7/31/2	<0.001

TABLE XVI
REAL-MODEL PILOT (6 PERSONAS, 72 CHANGE TURNS PER ROW, GATE WITH ORACLE REVIEWER). REACH: TARGET IN THE PROMPT; RIGHT OP. GIVEN REACH: CORRECT UPDATE AMONG TURNS WHERE THE TARGET WAS IN THE PROMPT; SUCCESS: CORRECT UPDATE ACCEPTED; STALE: FACTS STILL OUTDATED AT THE END (PER PERSONA).

Selection	Cand.	Reach	Right op. given reach	Success (95% CI)	Stale at end	Tokens/call
Default (12 matches, min. score)	10.4	0.67	0.91	0.61 (0.51–0.69)	4.7	645
Adaptive (F2)	23.0	0.93	0.91	0.85 (0.81–0.89)	1.8	890
Plain list of 24, no min. score	30.2	0.97	0.90	0.88 (0.78–0.96)	1.5	1033
Default + true target (oracle)	10.7	1.00	0.90	0.90 (0.85–0.96)	1.2	650

Table XVI shows three things. First, reachability carries through: success rose from 0.61 under the default to 0.85 (adaptive) and 0.88 (plain list of 24), and the number of outdated facts left at the end fell from 4.7 to 1.8 and 1.5 per persona. Second, once the target is in the prompt the model gets the update right about nine times in ten in every condition (0.91 to 0.91), and adding the true target to the default prompt gave 0.90 success; the remaining error, about one change turn in ten, is therefore not a selection problem in this setting. Third, the plain list of 24 was as good as the adaptive selection (0.88 against 0.85, with overlapping intervals) at about 1033 against 890 prompt tokens per call; with six personas we cannot say that either is better, and this agrees with the selection study, where the two were equal in reachability. The intervals of the adaptive, plain-list and oracle rows overlap; only the default differs clearly from the others.

X. RESULTS: ABLATION OF THE TWO SWITCHES

A. Simulated

Table XVII and Fig. 7 give the paired ablation under the gate (error sweep, $u = 0.05$, restating turns on). F2 reduces stale facts per persona at $e = 0.1$ from 1.83 to 0.65 and F1+F2 to 0.58; the F2 effects are significant ($p < 0.001$) in every cell shown. F1 alone moves F1 by +0.012 at $e = 0$ ($p = 0.031$), and is not significant at $e = 0.1$ and 0.2. With 210 tests that single p-value would not survive a correction. F2 raises the review load by about 5 prompts per 100 turns at $e = 0.1$ because more changes are discovered and proposed.

TABLE XVII
ABLATION UNDER THE GATE (SIMULATED). IN PARENTHESES: DIFFERENCE TO THE BASELINE AND SIGN-TEST p (60 PAIRS).

e	variant	F1 (diff., p)	stale (diff., p)	prompts/100
0	baseline	0.854	1.80	17.6
	+F1	0.865 (+0.012, 0.031)	1.70 (−0.10, 0.031)	18.3
	+F2	0.950 (+0.096, <0.001)	0.40 (−1.40, <0.001)	24.3
	F1+F2	0.967 (+0.114, <0.001)	0.27 (−1.53, <0.001)	25.0
0.1	baseline	0.817	1.83	23.7
	+F1	0.821 (+0.005, 0.500)	1.80 (−0.03, 0.500)	23.9
	+F2	0.890 (+0.074, <0.001)	0.65 (−1.18, <0.001)	29.1
	F1+F2	0.901 (+0.084, <0.001)	0.58 (−1.25, <0.001)	29.4
0.2	baseline	0.737	1.98	28.6
	+F1	0.747 (+0.010, 0.250)	1.92 (−0.07, 0.250)	28.8
	+F2	0.776 (+0.039, <0.001)	1.22 (−0.77, <0.001)	32.2
	F1+F2	0.784 (+0.047, <0.001)	1.17 (−0.82, <0.001)	32.7

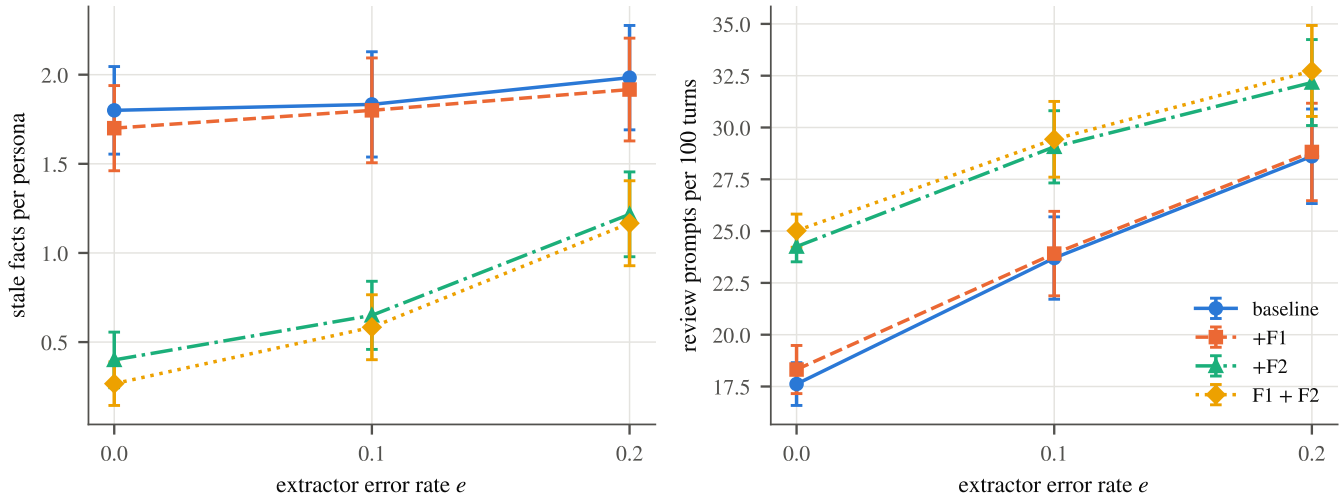


Fig. 7. Gate policy, error sweep: stale facts and review prompts for baseline, F1, F2, F1+F2 (simulated).

XI. RESULTS: REAL GEMINI EXTRACTORS

Table XVIII and Fig. 8 give the same ablation with a real extractor (12 personas, one seed, oracle user). Table XIX repeats baseline and F1+F2 with a second model.

Reachability on real conversations. With the baseline selection, the target memory was missing from the prompt on 25 of 136 change turns (first model; both policies together) and 22 of 136 (second model). With F1+F2 it was missing on 0 and 0. This is the mechanism of Sec. VIII observed with real conversations. The 12-persona real runs compared the default with F2 only; the same-budget list and an oracle-target condition were run later in a six-persona pilot (Sec. IX-A), in which the plain list matched F2 end to end.

Stale facts and F1. Under the gate, stale facts per persona fell from 1.00 to 0.00 (first model; $p = 0.002$) and from 1.08 to 0.00 (second; $p = 0.002$), and F1 rose from 0.905 to 0.982 (+0.077, $p = 0.021$) and from 0.845 to 0.901 (+0.057, $p = 0.002$). F2 alone reached 0.33 stale facts ($p = 0.021$) and F1 of 0.953 ($p = 0.146$, not significant); the benefit of adding F1 to F2 was not tested separately and is within what 12 personas can resolve. F1 alone changed nothing measurable (stale 1.08, $p = 1.000$): on real models F1 is not supported. Under auto-apply the picture is the same (stale 1.00 to 0.00, $p = 0.002$).

Safety on real models. No injected errors exist in real runs, so we count what the models did. Across all auto-apply persona runs (72), 7 true background facts were destroyed by the model's own operations (5 of them in the two baseline runs); across the 72 gate runs the count was 0. With the oracle user, no correct change was rejected, no wrong change was accepted and 0 wrong changes were applied. Because the user is perfect and the counts are small, this is a consistency check on the simulation, not a measurement of real users.

Cost. Prompt tokens per persona rose from about 13.4k to 18.1k (+35%, $p < 0.001$), and the number of prompts per 100 turns from 23.3 to 24.9 (first model, $p = 0.289$) and from 24.5 to 26.0 (second, $p = 0.180$), not significant. Of the change prompts, 68 of 69 (first model) and 68 of 72 (second) in the F1+F2 runs were correct changes that the user accepted, against 56 of 65 and 55 of 68 for the baselines; this is consistent with the added prompts being updates that the baseline missed.

Other effects. Under auto-apply the F2-only variant stored 3 false facts across 12 personas where the baseline stored none, and F1+F2 stored none again; with these counts we cannot say whether this is an effect of F2 or noise.

TABLE XVIII
REAL EXTRACTOR `gemini-3.5-flash-lite`: 12 PAIRED PERSONAS PER CELL. IN PARENTHESES: DIFFERENCE TO THE BASELINE AND EXACT SIGN-TEST p.

policy	variant	F1	stale	false	prompts/100	prompt tokens ($\times 10^3$)
auto	baseline	0.908	1.00	0.00	0.0	13.2
	+F1	0.902 (−0.007, 1.000)	1.00 (+0.00, 1.000)	0.08 (+0.08, 1.000)	0.0	13.2
	+F2	0.943 (+0.035, 0.453)	0.00 (−1.00, 0.002)	0.25 (+0.25, 0.250)	0.0	18.0
	F1+F2	0.956 (+0.048, 0.109)	0.00 (−1.00, 0.002)	0.00 (+0.00, 1.000)	0.0	18.0
gate	baseline	0.905	1.00	0.00	23.3	13.4
	+F1	0.897 (−0.008, 0.625)	1.08 (+0.08, 1.000)	0.00 (+0.00, 1.000)	21.9	13.3
	+F2	0.953 (+0.048, 0.146)	0.33 (−0.67, 0.021)	0.08 (+0.08, 1.000)	24.2	18.2
	F1+F2	0.982 (+0.077, 0.021)	0.00 (−1.00, 0.002)	0.00 (+0.00, 1.000)	24.9	18.1

TABLE XIX
SECOND EXTRACTOR `gemini-3.1-flash-lite`: BASELINE VERSUS F1+F2, 12 PAIRED PERSONAS.

policy	variant	F1	stale	false	prompts/100	prompt tokens ($\times 10^3$)
auto	baseline	0.840	0.92	0.00	0.0	13.2
	F1+F2	0.892 (+0.053, 0.012)	0.00 (−0.92, <0.001)	0.00 (+0.00, 1.000)	0.0	18.1
gate	baseline	0.845	1.08	0.00	24.5	13.3
	F1+F2	0.901 (+0.057, 0.002)	0.00 (−1.08, 0.002)	0.00 (+0.00, 1.000)	26.0	18.4

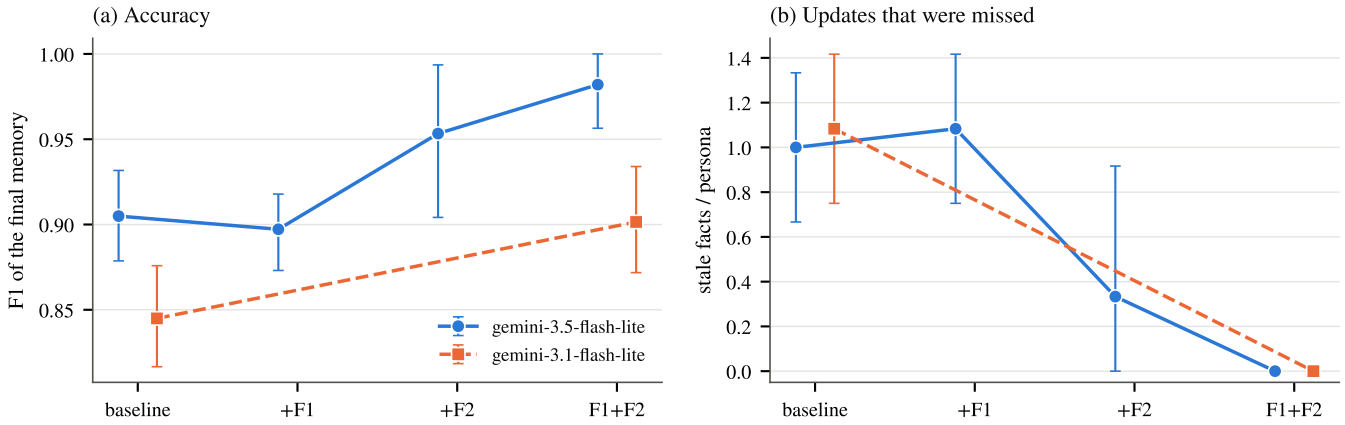


Fig. 8. Gate policy with real Gemini extractors: (a) F1, (b) stale facts per persona; mean and 95% bootstrap interval over 12 personas.

A. *Per-persona results*

Table XX lists every persona of the first model under the gate. With the baseline selection, 10 of 12 personas ended with at least one stale fact and 10 had at least one change target missing from the prompt; with F1+F2 none did. Persona 9 has the same F1 under both settings. Persona 3 has a lower F1 under F1+F2 because the model stored one more hypothetical sentence ("would like to live in Goa someday"), which lowers precision; none of its updates is wrong, so this is not a regression in updating.

B. *A worked example*

Persona 1 (Table IV) is a typical case. Table XXI shows the proposals made under the baseline, Table XXII the final memories, and Tables XXIII and XXIV the same under F1+F2. Under the baseline the extractor updated the gym time, the city (twice) and forgot the birthday, but it never proposed to replace "drives a Seltos"; the user's "I sold my old car and bought a Swift" became a separate fact ("bought a Swift car") and the Seltos memory remained active and stale. The one change target missing from the baseline prompt on this persona is consistent with this. A further proposal, replacing the project memory by "currently between projects", was rejected by the oracle reviewer. Under F1+F2 the same persona produced the car update, and the project memory was replaced by an "ended" record that was accepted.

The rejected proposal shows a limitation of our reviewer. The oracle judges proposals with the same value-based rules as the scorer, and a correct paraphrase can be rejected when the rules do not recognise its wording. In the baseline run this happened once for this persona; the final memory then still holds the old project memory, which the scorer rightly counts as stale. We therefore treat the oracle as an approximation of a careful user, not as one.

C. *Cost in requests, tokens and time*

Table XXV gives the cost per persona. The number of requests is the same (one extraction per user turn); prompt

TABLE XX
PER-PERSONA RESULTS, FIRST MODEL, GATE POLICY. MISSED TARGETS: CHANGE TURNS WHOSE TARGET MEMORY WAS NOT IN THE PROMPT.

	baseline			F1+F2		
persona	F1	stale	missed targets	F1	stale	missed targets
0	1.000	0	0	1.000	0	0
1	0.857	2	1	1.000	0	0
2	0.933	1	1	1.000	0	0
3	0.933	0	0	0.875	0	0
4	0.909	1	2	1.000	0	0
5	0.857	1	2	1.000	0	0
6	0.824	2	1	1.000	0	0
7	0.857	1	1	1.000	0	0
8	0.923	1	1	1.000	0	0
9	0.909	1	1	0.909	0	0
10	0.933	1	1	1.000	0	0
11	0.923	1	2	1.000	0	0
mean	0.905	1.00	1.08	0.982	0.00	0.00

TABLE XXI
PERSONA 1, BASELINE: PROPOSALS UNDER THE GATE.

session	action	proposal	accepted
3	update	The user goes to the gym at 7 am every day. → The user goes to the gym at 6 pm every day.	yes
3	update	The user is leading a project called Lotus at work. → The user is currently between projects at work as of October 2026.	no
4	forget	The user's birthday is on 21 September. → (removed)	yes
4	update	The user lives in Kolkata. → The user lives in Pune.	yes
5	update	The user lives in Pune. → The user lives in Chennai.	yes

TABLE XXII
PERSONA 1, BASELINE: FINAL PERSONA MEMORIES.

status	scored as	memory text
superseded	stale	The user goes to the gym at 7 am every day.
active	true	The user prefers their coffee as a cappuccino.
active	true	The user's dog is named Mithu.
superseded	stale	The user lives in Kolkata.
active	stale	The user drives a Seltos.
active	true	The user works at Swiggy.
active	stale	The user is leading a project called Lotus at work.
active	true	The user goes to the gym at 6 pm every day.
superseded	stale	The user lives in Pune.
active	true	The user bought a Swift car.
active	true	The user lives in Chennai.

TABLE XXIII
PERSONA 1, F1+F2: PROPOSALS UNDER THE GATE.

session	action	proposal	accepted
3	update	The user goes to the gym at 7 am every day. → The user goes to the gym at 6 pm every day.	yes
3	update	The user is leading a project called Lotus at work. → The user finished leading the project called Lotus and is currently between projects as of October 2026.	yes
4	forget	The user's birthday is on 21 September. → (removed)	yes
4	update	The user lives in Kolkata. → The user lives in Pune.	yes
5	update	The user lives in Pune. → The user lives in Chennai.	yes
5	update	The user drives a Seltos. → The user drives a Swift.	yes

TABLE XXIV
PERSONA 1, F1+F2: FINAL PERSONA MEMORIES.

status	scored as	memory text
superseded	stale	The user goes to the gym at 7 am every day.
active	true	The user drinks their coffee as a cappuccino.
active	true	The user's dog is named Mithu.
superseded	stale	The user lives in Kolkata.
superseded	stale	The user drives a Seltos.
active	true	The user works at Swiggy.
superseded	stale	The user is leading a project called Lotus at work.
active	true	The user goes to the gym at 6 pm every day.
active	ended	The user finished leading the project called Lotus and is currently between projects as of October 2026.
superseded	stale	The user lives in Pune.
active	true	The user lives in Chennai.
active	true	The user drives a Swift.

tokens grow by about 35% because more candidates are listed; output tokens are unchanged. Measured time per request is not a reliable cost measure here: it changed from 5.61 s to 9.39 s with the first model but from 6.15 s to 5.74 s with the second, which indicates that API load during the runs, not the prompt size, dominated.

D. Scorer validity

Table XXVI shows the agreement of the automatic labels with the author's hand labels. Three of 195 + 175 memories differ; in all three the automatic scorer labelled a correct statement of unemployment or finished project as "ended" or "other" while the annotator judged it true, so the automatic F1 is, if anything, slightly conservative on these three. These labels were kept as annotated. Since the scorer and the labels share one author and the same persona rules, the agreement shows consistency, not independence.

TABLE XXV
COST PER PERSONA (GATE POLICY; MEANS OVER 12 PERSONAS).

model	variant	requests	prompt tokens	output tokens	latency / request (s)
`gemini-3.5-flash-lite`	baseline	23.1	13 376	833	5.61
	F1+F2	23.1	18 072	796	9.39
`gemini-3.1-flash-lite`	baseline	23.1	13 347	932	6.15
	F1+F2	23.1	18 394	939	5.74

TABLE XXVI
AGREEMENT BETWEEN AUTOMATIC AND HAND LABELS (SINGLE ANNOTATOR).

run	labelled memories	agreement	rate	Cohen's κ
baseline	195	193	0.990	0.975
F1+F2	175	174	0.994	0.981

XII. EXTERNAL CHECK: LONGMEMEVAL

LongMemEval's knowledge-update questions ask for the latest value of a fact that changed during the history [2]. We took 72 such questions of the "S" setting and played each with all evidence sessions plus three random distractor sessions (the same for every system) through AURA's extractor, store and search; answers come from the ten memories that AURA's search returns, and are graded by a Gemini judge written after the benchmark's rule (the official prompt is not reproduced) and by string match. All runs use `gemini-3.5-flash-lite`. There is no label for which memory changes are right, so the gate's reviewer is an oracle approximation: it accepts proposals made while reading an evidence session and rejects the rest. This gate figure is therefore an upper-bracket, not a measurement of real users. Baselines are answering with no memory and with the raw sessions as context.

Table XXVII and Fig. 9 show no measurable effect of the F1+F2 selection on this benchmark. The gate system scores 0.444 in the baseline run and 0.444 in the F1+F2 run (5 questions worse, 5 better, $p = 1.000$); append-only and auto-apply also do not differ between the runs (1.000 and 0.648). The systems without memory or with full context, which the selection cannot affect, differ by +0.028 between the two runs, which indicates the run-to-run noise of the answering and judging model. A smaller earlier run on the first 30 of these questions had suggested a gain for the gate (0.267 to 0.433, $p = 0.063$); on all 72 questions it is absent, and we regard the earlier difference as noise. Full-context answering (0.847 in the second run) remains far above the gate system (0.444; 31 vs. 2 discordant questions, $p =< 0.001$): a memory system trades accuracy for the properties it provides, such as small prompts, forgetting and user control. Three reasons make this benchmark a weak test of the selection: each question is played with only a handful of sessions, so the store holds about ten memories and the default selection can already show nearly all of them; answers are limited by what the extractor stores as much as by what it updates (the reference answer was in an active memory for only 0.458 of questions under the gate); and the gate's reviewer is an oracle approximation. It therefore shows that the selection does not hurt, not that it helps.

TABLE XXVII
LONGMEMEVAL KNOWLEDGE-UPDATE QUESTIONS (JUDGE-GRADED ACCURACY, $n = 72$). b/c: QUESTIONS CORRECT ONLY IN THE BASELINE / ONLY IN THE F1+F2 RUN.

system	baseline	F1+F2	difference	b/c	p
no memory	0.000	0.000	+0.000	0/0	1.000
full context	0.819	0.847	+0.028	0/2	0.500
append-only	0.361	0.361	+0.000	12/12	1.000
auto-apply	0.472	0.431	−0.042	11/8	0.648
gate (oracle reviewer)	0.444	0.444	+0.000	5/5	1.000

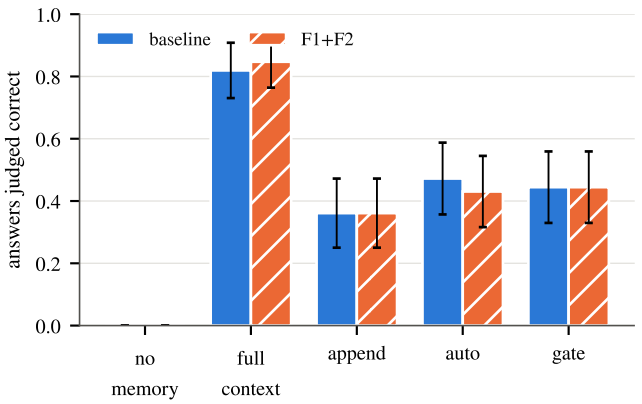


Fig. 9. LongMemEval accuracy by system for baseline and F1+F2 selection (95% normal-approximation bars).

XIII. Discussion

Where the guarantee ends. The gate provides a guarantee about what is *written*: nothing changes without approval. It provides none about what is *proposed*. In our experiments the second quantity depends on a component that looks like plumbing, the choice of candidate memories, and with default settings it is the main reason that stale facts remain. Evaluating a user-controlled memory by its write accuracy or by the number of approvals alone would miss this.

Design implications. (i) Report reachability alongside accuracy for any extractor-based memory, on change events and on the final prompt. (ii) For the extractor's candidate search, the minimum score is the largest single loss; a search tuned to keep unrelated memories out of *answers* is the wrong default for choosing what an update may touch. (iii) Search per sentence when messages are mixed. (iv) Do not rely on cue words: in our second test set they added almost nothing on their own and made the adaptive mode weaker on messages phrased without them; a plain larger list did as well at a somewhat larger prompt, and reranking that list with a cross-encoder did better. (v) Budget for more prompt tokens (about 35% with F2 in the real runs) and slightly more review cards.

What the gate is for. The gate is a user-control mechanism as much as a safety mechanism. Every change to what the assistant believes about a person passes a visible decision, old values remain as history, and forgetting is an explicit operation that removes the memory. Our results show what this buys (protection against wrong writes) and what it costs (20–33 prompts per 100 turns in simulation, a little more with F2). They also show a limit: the gate's protection is only as good as the user's attention (Table VI), and adding prompts can lower that attention. Whether F2's additional prompts, which are mostly correct updates, are better tolerated than the baseline's is an empirical question for a user study.

Why the missed-update problem is easy to overlook. A missed update does not produce an error message, a wrong answer on the turn it occurs, or a prompt. It surfaces later, when the assistant states an outdated fact. Evaluations that look at single-turn extraction, or at accuracy of what is written, will not see it; measuring the final memory state against the true state, as we do, does.

Choosing between larger prompts and better search. F2 enlarges the prompt by about a third, and in the selection study a plain list of the same size reached as many targets. Keyword and fused rankings did no better; a cross-encoder reranker over the 50 best matches did, by about four points at the same list size, at the cost of one reranking pass per message. About one target in thirteen at $N = 350$ was out of reach of every method we tried. Closing that gap needs a different mechanism: a second extraction pass over the whole store when a change is suspected, or a structured memory with typed slots in which an update names its slot. A learned admission gate [8] could make larger lists safer but cannot raise reachability on its own. We did not evaluate these; the point is that reachability is a quantity worth tracking whichever remedy is chosen, and that simple remedies should be compared against plain larger lists before they are credited.

On F1. Letting a rejected change return only after new user input is cheap and conceptually neat, but we found no benefit with real models (the oracle user rejected no correct change in these runs), and only small, uncorrected-significance effects in simulation. Its value would have to be shown with real users who slip.

XIV. Threats to Validity

Synthetic and English personas. Personas, unrelated memories and the wording of changes were written by the authors from templates. The change cues of F2 were designed after the reachability failures were seen on the first generator, so F2 may be tuned to it. The second test set (Sec. IX) reduces this risk with a held-out, cue-free half, and it shows that F2 does depend on the cue words; but it is still template-based and written by the same team, and the LongMemEval check is the only independent text. **Small real samples.** The oracle-target and same-size-list pilot has six personas, one model and one seed, after two planned personas were lost to an API quota error. 12 personas, one seed per cell and two models of the same family; exact sign tests with 12 pairs cannot resolve small effects, and several F1 results are inconclusive rather than null. **Simulated user.** The real runs use a perfect reviewer, the simulations a random slip. Real users may ignore, batch or rubber-stamp prompts; whether additional prompts are tolerated is open. No user study was done. **Single annotator and scorer.** Labels come from the author. **One embedding model.** Reachability depends on MiniLM and the 0.25 threshold. **LongMemEval.** 72 questions (all non-abstention knowledge-update questions of the cleaned file), a judge that is not the official prompt, an oracle reviewer, and a single seed. **Multiple tests.** Reported as counts; only the large effects are robust to correction. **Related work.** Six of the closest works were read in full, the others at abstract level; the search was targeted, not systematic.

XV. Future Work

User study. The most important missing piece is a study with real users that measures how often they approve, reject or ignore proposals, whether they slip, and whether the extra prompts from F2 are tolerated. **Isolating each factor with real models.** The pilot of Sec. IX-A used six personas and one model; repeating it with all planned personas, several seeds and a second model family, with the failed runs re-done, would turn its suggestive result into a finding. **Better candidate selection.** About one target in thirteen is out of reach of every method we tried, a cross-encoder reranker included; per-sentence reranking, slot-typed memories and a second pass over the store are the next candidates. **Other languages and models.** All personas are English and both extractors are Gemini models. **Bigger real samples.** More personas, more seeds, would turn several inconclusive results into findings; the LongMemEval check already covers every knowledge-update question of the file we used, and a harder setting with larger stores would test the selection better. **Proactive use of memory.** AURA also contains an optional layer that asks the user whether a stored fact is still true, sends reminders and prepares briefings; its checks use the same proposal path as the gate. We did not evaluate it and make no claim about it here.

XVI. Conclusion

A review gate on memory changes is effective against the errors it can see: in our experiments it removes almost all of the true facts that automatic application destroys. It cannot repair updates the extractor never proposes, and in a retrieval-based design these are governed by which memories reach the extractor's prompt. We made this precise (an update can succeed only if its target is in the prompt, and the gate acts only after that), measured it, and identified what moves it: removing the minimum score and searching per sentence each raise reachability by about ten points, extra candidates on a change cue add little on their own, and an adaptive selection is matched by a plain larger list, which it beats only in prompt size. With two real Gemini extractors, the adaptive selection (run together with the re-proposal rule) left no change target missing and no stale fact under the gate, at about a third more prompt tokens, while keyword and fused rankings did no better than a plain larger list, a cross-encoder reranker did better at the same list size, and about one target in thirteen remains out of reach of every method tried. The practical lesson for user-controlled memories is to measure and report what the extractor can reach, and to compare any remedy against a plain larger list. Evaluating with real users, with an oracle-target condition, in other languages and with larger real samples is the next step; the code, both test-set generators and every raw result file are provided.

Data and Code Availability

The evaluation harness (`eval/`), both persona generators, all raw per-run files and the scripts that produce every table and figure of this paper (`eval/make_paper_assets.py` and `eval/selection/make_paper_tables.py`) are in the AURA project repository. Numbers in the text are generated from the result folders.

Appendix A
Scoring Rules and Labels

Value matching. Each persona slot has a set of possible values (for example the gym slot has "6 am", "7 am", "6 pm", "8 pm"), each with alternative spellings ("6am", "6:00 am", "18:00"); birthdays accept "12 March", "March 12", "12th March" and so on; coffee values accept "black coffee" or "oat milk". A value is matched as a whole word, case-insensitively. Distractor values (a hypothetical city, a friend's car) have their own matchers.

Wording of third parties, hypotheticals and ended facts. A memory that mentions only a distractor value is *third-party* if it contains a word for a relative, friend, colleague or neighbour, *hypothetical* if it contains "if", "would" or "hypothetical", and *false* otherwise. A memory about a retired fact is *ended* if it contains a phrase such as "no longer", "finished", "wrapped up", "between projects", "stopped", "sold", "unemployed" or "not working" and the fact was retired but not asked to be forgotten. A fact that the user asked to forget and that is still stored counts as *stale*.

Annotation protocol. For every memory in the baseline and F1+F2 real runs the annotator saw the persona's sessions, the true final state and the memory text, and wrote one of seven labels (true, ended, stale, false, hypothetical, third-party, other), looking at the automatic label only afterwards where possible. The automatic and the hand label were then compared (Table XXVI).

Appendix B
Complete Policy Results

Table XXVIII gives all metrics of the policy comparison for all error rates.

TABLE XXVIII
All policies, all extractor error rates (simulated, $u = 0.05$; mean per persona).

e	policy	F1	precision	recall	stale	false	destroyed	wrong applied	prompts/100
0	append-only	0.597	0.483	0.791	5.00	0.00	0.00	0.00	0.0
	auto-apply	0.881	0.798	0.992	1.57	0.00	0.00	0.00	0.0
	gate changes	0.854	0.768	0.968	1.80	0.00	0.00	0.00	19.5
	gate all	0.861	0.788	0.959	1.62	0.00	0.00	0.00	65.8
0.05	append-only	0.584	0.474	0.769	4.90	0.12	0.00	0.00	0.0
	auto-apply	0.840	0.765	0.944	1.65	0.17	0.78	0.82	0.0
	gate changes	0.820	0.741	0.929	1.90	0.12	0.05	0.07	22.2
	gate all	0.818	0.753	0.908	1.82	0.00	0.03	0.05	67.3
0.1	append-only	0.581	0.478	0.753	4.58	0.33	0.00	0.00	0.0
	auto-apply	0.798	0.730	0.898	1.68	0.43	1.60	1.75	0.0
	gate changes	0.797	0.725	0.903	1.85	0.33	0.03	0.05	24.4
	gate all	0.795	0.749	0.867	1.83	0.02	0.05	0.05	69.0
0.2	append-only	0.549	0.457	0.703	4.25	0.70	0.00	0.00	0.0
	auto-apply	0.735	0.675	0.831	1.73	0.83	2.83	3.10	0.0
	gate changes	0.733	0.666	0.836	1.98	0.70	0.18	0.23	28.8
	gate all	0.765	0.732	0.818	1.88	0.00	0.13	0.17	72.5
0.3	append-only	0.523	0.440	0.665	3.87	1.20	0.00	0.00	0.0
	auto-apply	0.650	0.596	0.743	1.92	1.33	4.60	4.97	0.0
	gate changes	0.653	0.585	0.765	2.18	1.22	0.25	0.28	33.4
	gate all	0.707	0.700	0.743	1.98	0.07	0.40	0.43	77.6

Appendix C
Complete Ablation Results

Tables XXIX and XXX give every cell of the simulated ablation. An asterisk marks a sign-test $p < 0.05$ against the baseline (uncorrected; 210 tests in all). The upper three blocks vary the extractor error rate ($u = 0.05$), the lower four the user slip rate ($e = 0.1$; gate only).

TABLE XXIX
ABLATION, GATE POLICY: ALL CELLS (60 PAIRED PERSONA RUNS EACH). * : UNCORRECTED $p < 0.05$.

e	u	variant	F1	stale	false	exact state	prompts/100
0	0.05	baseline	0.854	1.80	0.00	0.033	17.6
		+F1	0.865*	1.70*	0.00	0.050	18.3*
		+F2	0.950*	0.40*	0.00	0.667*	24.3*
		F1+F2	0.967*	0.27*	0.00	0.750*	25.0*
0.1	0.05	baseline	0.817	1.83	0.33	0.067	23.7
		+F1	0.821	1.80	0.33	0.067	23.9
		+F2	0.890*	0.65*	0.33	0.267*	29.1*
		F1+F2	0.901*	0.58*	0.33	0.300*	29.4*
0.2	0.05	baseline	0.737	1.98	0.80	0.017	28.6
		+F1	0.747	1.92	0.80	0.017	28.8
		+F2	0.776*	1.22*	0.82	0.083	32.2*
		F1+F2	0.784*	1.17*	0.82	0.083	32.7*
0.1	0	baseline	0.826	1.75	0.33	0.083	23.7
		+F1	0.826	1.75	0.33	0.083	23.8
		+F2	0.900*	0.57*	0.33	0.350*	29.1*
		F1+F2	0.903*	0.55*	0.33	0.367*	29.2*
0.1	0.1	baseline	0.801	1.98	0.35	0.050	23.6
		+F1	0.809	1.92	0.35	0.050	24.2*
		+F2	0.861*	0.97*	0.33	0.233*	28.9*
		F1+F2	0.881*	0.82*	0.33	0.267*	29.9*
0.1	0.2	baseline	0.771	2.32	0.35	0.000	23.3
		+F1	0.793*	2.20*	0.33	0.000	24.5*
		+F2	0.829*	1.25*	0.37	0.050	28.6*
		F1+F2	0.851*	1.10*	0.37	0.050	30.1*
0.1	0.3	baseline	0.740	2.60	0.38	0.017	23.3
		+F1	0.750	2.53	0.38	0.017	25.3*
		+F2	0.765*	1.80*	0.35	0.033	28.6*
		F1+F2	0.800*	1.57*	0.33	0.033	31.0*

TABLE XXX
ABLATION, AUTO-APPLY POLICY: ERROR-RATE CELLS (60 PAIRED PERSONA RUNS EACH). * : UNCORRECTED $p < 0.05$.

e	u	variant	F1	stale	false	exact state	prompts/100
0	0.05	baseline	0.881	1.57	0.00	0.083	0.0
		+F1	0.881	1.57	0.00	0.083	0.0
		+F2	0.989*	0.08*	0.00	0.917*	0.0
		F1+F2	0.989*	0.08*	0.00	0.917*	0.0
0.1	0.05	baseline	0.819	1.62	0.40	0.000	0.0
		+F1	0.819	1.62	0.40	0.000	0.0
		+F2	0.899*	0.43*	0.42	0.050	0.0
		F1+F2	0.899*	0.43*	0.42	0.050	0.0
0.2	0.05	baseline	0.750	1.62	0.93	0.000	0.0
		+F1	0.750	1.62	0.93	0.000	0.0
		+F2	0.800*	0.88*	0.87	0.000	0.0
		F1+F2	0.800*	0.88*	0.87	0.000	0.0

APPENDIX D
REACHABILITY FOR ALL STORE SIZES

Table XXXI gives reachability and mean candidate count for all selection modes, store sizes and turn types.

TABLE XXXI
REACHABILITY (MEAN NUMBER OF CANDIDATES) BY MODE, STORE SIZE N AND TURN TYPE.

N	turn type	default	no min.	sentence	adaptive
0	single sentence	0.898 (8.2)	1.000 (11.4)	0.898 (8.2)	1.000 (11.4)
	mixed turn	0.747 (8.1)	0.998 (11.4)	0.898 (8.2)	1.000 (11.9)
25	single sentence	0.752 (8.8)	0.990 (17.4)	0.752 (8.8)	0.993 (23.8)
	mixed turn	0.439 (8.2)	0.903 (17.4)	0.752 (8.8)	0.980 (24.2)
50	single sentence	0.749 (9.1)	0.975 (18.5)	0.749 (9.1)	0.986 (25.9)
	mixed turn	0.364 (8.3)	0.822 (18.5)	0.749 (9.1)	0.969 (25.9)
100	single sentence	0.727 (9.6)	0.952 (19.2)	0.727 (9.6)	0.969 (27.1)
	mixed turn	0.344 (8.3)	0.745 (19.3)	0.727 (9.6)	0.921 (27.2)
200	single sentence	0.704 (10.4)	0.934 (19.6)	0.704 (10.4)	0.951 (27.8)
	mixed turn	0.305 (8.3)	0.668 (19.5)	0.704 (10.4)	0.900 (27.8)
350	single sentence	0.690 (11.3)	0.911 (19.8)	0.690 (11.3)	0.936 (28.2)
	mixed turn	0.286 (8.3)	0.607 (19.8)	0.690 (11.3)	0.866 (28.1)

APPENDIX E
SELECTION STUDY FOR ALL STORE SIZES

Table XXXII gives reachability for every condition of the selection study and every store size, with the retrieval diagnostics of Sec. IX.

TABLE XXXII
SELECTION STUDY: REACHABILITY BY CONDITION AND STORE SIZE N, AND DIAGNOSTICS (3,200 CHANGE EVENTS PER STORE SIZE).

condition	$N = 0$	$N = 25$	$N = 50$	$N = 100$	$N = 200$	$N = 350$
default	0.897	0.682	0.682	0.682	0.680	0.667
no minimum	0.999	0.959	0.925	0.883	0.830	0.777
per sentence	0.929	0.787	0.787	0.786	0.779	0.767
adaptive (F2)	1.000	0.982	0.955	0.921	0.877	0.845
per sentence + no min.	1.000	0.969	0.940	0.902	0.856	0.825
k=24 on cue	0.897	0.682	0.682	0.682	0.681	0.674
no min. + k=24 on cue	0.999	0.978	0.943	0.904	0.856	0.812
per sentence + k=24 on cue	0.929	0.787	0.787	0.786	0.782	0.773
default, k=24	0.897	0.682	0.682	0.682	0.682	0.680
no min., k=24	0.999	0.991	0.969	0.934	0.888	0.849
per sentence + no min., k=24	1.000	0.993	0.978	0.947	0.907	0.870
BM25, k=12	0.907	0.728	0.728	0.728	0.728	0.726
BM25, k=24	0.907	0.728	0.728	0.728	0.728	0.728
fusion (RRF), k=24	1.000	0.991	0.970	0.936	0.882	0.843
per sentence + fusion, k=24	1.000	0.992	0.979	0.950	0.903	0.861
target in top 12, no floor	1.000	0.959	0.925	0.883	0.830	0.777
target in top 24, no floor	1.000	0.991	0.969	0.934	0.888	0.849
in top 12 but below 0.25	0.318	0.277	0.243	0.201	0.150	0.110
missed by all 11 conditions	0.000	0.004	0.014	0.036	0.073	0.100

APPENDIX F
REPRODUCING THE RESULTS

All commands are run from the repository root. Simulated experiments need no API key; real-model experiments need `GEMINI_API_KEY` and internet access.

- `npm run eval:gate`, `eval:retrieval`, `eval:reach`: policies, retrieval, reachability;
- `npm run eval:gate -- --errors 0,0.1,0.2 --policies auto,gate --restate 1 --candidates adaptive --repropose 2 --out eval/results/ablation/both` (and the three other variants), then `npm run eval:compare` for the paired statistics;
- `npm run eval:real -- --yes`: real-model runs (twelve personas, four variants, one folder each); `npm run eval:rescore`: scoring and annotation sheet;
- `npm run eval:lme -- --yes`: LongMemEval subset; `npm run eval:compare-lme`;

- `node eval/selection/run-reach.mjs`, `python3 eval/selection/analyze.py`, `python3 eval/selection/make_paper_tables.py`: the selection study (no API key; tests in `eval/selection/tests`);
- `python eval/make_paper_assets.py --root eval --out paper`: every other table, number macro and the real-run and LongMemEval figures of this paper.

REFERENCES

[1] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," *arXiv preprint arXiv:2402.17753*, 2024.

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," *arXiv preprint arXiv:2410.10813*, 2024.

[3] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," *arXiv preprint arXiv:2504.19413*, 2025.

[4] T. Yang, S. Paul, V. Srinivasan, V. Kulkarni, and S. Chappidi, "TrustMem: Learning trustworthy memory consolidation for LLM agents with long-term memory," *arXiv preprint arXiv:2606.25161*, 2026.

[5] Y. Yu, L. Yao, Y. Xie, Q. Tan, J. Feng, Y. Li, and L. Wu, "Agentic memory: Learning unified long-term and short-term memory management for large language model agents," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 21 457–21 483.

[6] V. Patel, "Supersede: Diagnosing and training the memory-update gap in LLM agents," *arXiv preprint arXiv:2606.27472*, 2026.

[7] Y. Song, Y. Wang, X. Ma, Z. Fu, J. Lin, W. Liu, J. Wang, H. Deng, Y. Yu, and W. Zhang, "Retrieval-driven memory reconsolidation for long-term LLM agents," *arXiv preprint arXiv:2609.16053*, 2026.

[8] J. Zhang, K. Chen, J. Ma, Y. Hu, L. He, Y. Zhang, J. Liu, X. Yang, T. Zhang, and R. Jia, "Beyond similarity: Trustworthy memory search for personal AI agents," *arXiv preprint arXiv:2606.06054*, 2026.

[9] C. Lam, J. Li, L. Zhang, and K. Zhao, "Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework," *arXiv preprint arXiv:2603.11768*, 2026.

[10] Z. Xiong, Y. Lin, W. Xie, P. He, Z. Liu, J. Tang, H. Lakkaraju, and Z. Xiang, "How memory management impacts LLM agents: An empirical study of experience-following behavior," *arXiv preprint arXiv:2505.16067*, 2025.

[11] S. Xiao, S. Wang, X. Chen, K. Chao, M. Cui, F. Qian, F. Meng, C. Mei, C. Jiang, Q. Ouyang, and J. Yi, "When errors become memories: Causal pathway tracing in multi-turn memory-augmented LLMs," *arXiv preprint arXiv:2608.30198*, 2026.

[12] H. Sun, Z. Zhang, and S. Zeng, "Preference-aware memory update for long-term LLM agents," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 783–793.

[13] K. Chu, "A systematic survey of security threats and defenses in LLM-based AI agents: A layered attack surface framework," *arXiv preprint arXiv:2604.23338*, 2026.

[14] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," *arXiv preprint arXiv:2002.10957*, 2020.

[15] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[16] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.